計算機科学実験及演習 3 ハードウェア

機能設計仕様書

タイマ・I/O 拡張・7セグ改善・応用プログラム

担当 B — 周辺回路・ソフトウェア設計

学籍番号　（学籍番号を記入）

氏名　（氏名を記入）

提出日　2026 年 5 月 8 日

京都大学 工学部情報学科 計算機科学コース

目次

1 担当範囲 2

2 タイマモジュール 2
- 2.1 設計方針 2
- 2.2 レジスタマップ 2
- 2.3 CTRL レジスタのビットフィールド 2
- 2.4 動作仕様 2
- 2.5 RTL 実装 3

3 I/O アドレス空間の拡張 5
- 3.1 設計方針 5
- 3.2 CPU の変更 6
- 3.3 I/O ポートマップ 6
- 3.4 入力デコーダ 6
- 3.5 出力デコーダ 7

4 7 セグメント表示の改善 7
- 4.1 8 桁対応 7
- 4.2 表示モード切替 8
- 4.3 デバッグポート 8

5 応用プログラム: バブルソート 9
- 5.1 概要 9
- 5.2 アルゴリズム 9
- 5.3 レジスタ割り当て 9
- 5.4 プログラムリスティング 9
- 5.5 プログラムサイズと実行サイクル数 10
- 5.6 シミュレーション結果 10

6 アセンブラ 11
- 6.1 概要 11
- 6.2 対応命令 11
- 6.3 アセンブラの機能 12
- 6.4 使用方法 12

7 動作検証 12
- 7.1 テスト環境 12
- 7.2 ISA 全命令テスト 12
- 7.3 バブルソートテスト 13

7.4 テストベンチの設計 13

8 **まとめ** 13

1 担当範囲

本書は担当 B の設計・実装範囲について報告する。

- **タイマモジュール**: ハードウェアタイマカウンタの設計と実装
- **I/O アドレス空間の拡張**: 複数デバイスを切り替え可能な I/O ポートデコーダの設計
- **7 セグメント表示の改善**: 8 桁フル表示と表示モード切替
- **応用プログラム**: バブルソートのアセンブリプログラム
- **アセンブラ**: 全命令対応の Python アセンブラ

担当 A が実装した 4 サイクル CPU コアおよび ISA 拡張（BAL/BR/ADDI）の上に、本担当の周辺回路・ソフトウェアを統合することで、完成度の高いプロセッサシステムを構築する。

2 タイマモジュール

2.1 設計方針

ソフトウェア遅延ループに依存しない正確な時間計測を実現するため、ハードウェアタイマカウンタを設計した。プリスケーラにより 20MHz のクロックを任意の周波数に分周し、16 ビットカウンタでイベント計数・タイムアウト検出を行う。

2.2 レジスタマップ

タイマは 3 つのレジスタを持ち、I/O ポート 2・3 経由でアクセスする。

表 1 タイマレジスタマップ

I/O ポート	レジスタ	R/W	機能
2 (IN)	COUNT	R	現在のカウント値 (16 ビット)
2 (OUT)	CTRL	W	制御レジスタ
3 (IN)	overflow	R	bit0: カウント到達フラグ (Read-to-Clear)
3 (OUT)	LIMIT	W	カウント上限値

2.3 CTRL レジスタのビットフィールド

2.4 動作仕様

1. `CTRL.EN = 1` のとき、プリスケーラカウンタが毎クロック +1 される
2. プリスケーラカウンタが `CTRL[15:8]` に到達すると 0 に戻り、メインカウンタ（COUNT）を +1 する
3. COUNT が LIMIT に到達すると:
   - `overflow` フラグが 1 になる

表 2　CTRL レジスタ構成

ビット	名前	機能
0	EN	1: カウント動作, 0: 停止
1	RESET	1: COUNT を 0 にリセット (ワンショット, 自動クリア)
7:2	(予約)	未使用
15:8	PRESCALER	プリスケーラ分周値

- COUNT が 0 にリセットされる

4. `overflow` は IN 命令でポート 3 を読み出すと自動クリアされる
5. `CTRL.RESET` に 1 を書くと COUNT が即座に 0 になり、次サイクルで自動的に RESET ビットがクリアされる

2.4.1　プリスケーラによる周波数分周

カウント周波数は以下の式で決まる:

$$f_{\text{count}} = \frac{f_{\text{clk}}}{\text{PRESCALER} + 1} = \frac{20\,\text{MHz}}{\text{PRESCALER} + 1}$$

表 3　プリスケーラ設定例

PRESCALER 値	カウント周波数	用途
0	20 MHz	高精度計測 (50ns 分解能)
19	1 MHz	マイクロ秒単位の計測
199	100 kHz	10μs 分解能、約 0.65 秒でオーバーフロー
255	約 78 kHz	最大分周

2.5　RTL 実装

```
// timer.v - ハードウェアタイマカウンタ (担当B)
//
// レジスタマップ (I/Oポート経由でアクセス):
//   COUNT (R):   現在のカウント値 (16bit)
//   CTRL  (R/W): bit0=動作/停止, bit1=リセット(ワンショット), bit[15:8]=プリスケーラ値
//   LIMIT (R/W): カウント上限値。到達でoverflow=1, COUNT=0
//
// プリスケーラ: clk / (prescaler + 1) の周波数でCOUNTをインクリメント
//   prescaler=0:   20MHz (50ns間隔)
//   prescaler=199: 100kHz (10us間隔)

module timer(
```

```
    input          clk,
    input          rst_n,
    input  [15:0]  wdata,      // 書き込みデータ
    input          we_ctrl,    // CTRL レジスタ書き込み
    input          we_limit,   // LIMIT レジスタ書き込み
    input          rd_overflow, // overflow 読み出し（クリア用）
    output [15:0]  count,      // 現在のカウント値
    output reg     overflow    // カウント到達フラグ
);

// ---- レジスタ ----
reg [15:0] count_reg;
reg [15:0] limit_reg;
reg [15:0] ctrl_reg;

assign count = count_reg;

// CTRL フィールド
wire        timer_en   = ctrl_reg[0];   // 動作/停止
wire [7:0]  prescaler  = ctrl_reg[15:8]; // プリスケーラ値

// ---- プリスケーラカウンタ ----
reg [7:0] pre_cnt;

always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        count_reg <= 16'h0000;
        limit_reg <= 16'hFFFF;
        ctrl_reg  <= 16'h0000;
        pre_cnt   <= 8'd0;
        overflow  <= 1'b0;
    end else begin
        // CTRL 書き込み
        if (we_ctrl) begin
            ctrl_reg <= wdata;
            // bit1 リセット: COUNT を 0 にクリア（ワンショット）
            if (wdata[1]) begin
                count_reg <= 16'h0000;
                pre_cnt   <= 8'd0;
                overflow  <= 1'b0;
            end
        end

        // CTRL.bit1 自動クリア（書き込みの次サイクル）
        if (ctrl_reg[1] && !we_ctrl)
            ctrl_reg[1] <= 1'b0;

        // LIMIT 書き込み
        if (we_limit)
```

```
            limit_reg <= wdata;

        // overflow 読み出しでクリア
        if (rd_overflow)
            overflow <= 1'b0;

        // タイマ動作
        if (timer_en && !ctrl_reg[1]) begin
            if (pre_cnt >= prescaler) begin
                pre_cnt <= 8'd0;
                // メインカウンタ インクリメント
                if (count_reg >= limit_reg) begin
                    count_reg <= 16'h0000;
                    overflow  <= 1'b1;
                end else begin
                    count_reg <= count_reg + 16'd1;
                end
            end else begin
                pre_cnt <= pre_cnt + 8'd1;
            end
        end
    end
end

endmodule
```

Listing 1　timer.v — タイマモジュール

2.5.1　実装のポイント

- プリスケーラカウンタ（8 ビット）とメインカウンタ（16 ビット）の 2 段構成
- CTRL.RESET のワンショット動作: 書き込みの次サイクルで自動クリア
- overflow フラグの Read-to-Clear: IN 命令でポート 3 を読み出すと `rd_overflow` 信号が立ち、次サイクルで overflow が 0 にクリアされる
- 全レジスタは非同期リセット（`rst_n`）対応

3　I/O アドレス空間の拡張

3.1　設計方針

基本設計の IN/OUT 命令では単一の入出力先に固定されており、複数の I/O デバイスを接続できない。IN/OUT 命令の `d` フィールド下位 4 ビット（`I[3:0]`）を I/O ポート番号として利用し、最大 16 デバイスのアドレッシングを可能にする。

3.2 CPU の変更

simple_cpu.v に以下の 3 つの出力ポートを追加した。

表 4 追加した CPU 出力ポート

信号名	幅	機能
io_port	4 bit	I/O ポート番号 (d4 = I[3:0])
io_read	1 bit	IN 命令実行中 (PH4 かつ IN 命令)
io_write	1 bit	OUT 命令実行中 (PH4 かつ OUT 命令)

```
assign io_port  = d4;  // I[3:0] をそのまま出力
assign io_read  = (phase == PH4) && running && is_in;
assign io_write = (phase == PH4) && running && is_out;
```

Listing 2 I/O ポート信号の生成

これにより、アセンブリ側では IN r0, 2 や OUT r1, 3 のようにポート番号を指定して I/O 操作を行える。ポート番号 0 を指定した場合は従来と同じ動作となり、後方互換性が保たれる。

3.3 I/O ポートマップ

表 5 I/O ポートマップ

ポート	IN (読み出し)	OUT (書き込み)	備考
0	DIP スイッチ A (SW26, 8bit)	7 セグ出力 (下位 16bit)	従来互換
1	DIP スイッチ B (SW27, 8bit)	7 セグ出力 (上位 16bit)	8 桁拡張用
2	タイマ COUNT 値	タイマ CTRL レジスタ	
3	タイマ overflow (bit0)	タイマ LIMIT レジスタ	
4	プッシュスイッチ (4bit)	LED 出力 (8bit)	
5	ロータリースイッチ (4bit)	(予約)	表示モード用

3.4 入力デコーダ

IN 命令実行時、io_port に応じて適切なデータソースを in_data に接続する。

```
always @(*) begin
    case (io_port)
        4'd0:    in_mux = {8'h00, ~sw[7:0]};
        4'd1:    in_mux = {8'h00, ~sw[15:8]};
        4'd2:    in_mux = timer_count;
        4'd3:    in_mux = {15'd0, timer_overflow};
        4'd4:    in_mux = {12'h000, ~pushsw};
```

```
        4'd5:     in_mux = {12'h000, rotsw};
        default: in_mux = 16'h0000;
    endcase
end
```

Listing 3　I/O 入力マルチプレクサ

3.5　出力デコーダ

OUT 命令実行時、io_port に応じて適切なレジスタに書き込む。

```
always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        seg_reg_lo <= 16'h0000;
        seg_reg_hi <= 16'h0000;
        led        <= 8'h00;
    end else if (io_write) begin
        case (io_port)
            4'd0: seg_reg_lo <= cpu_out_data;
            4'd1: seg_reg_hi <= cpu_out_data;
            4'd4: led        <= cpu_out_data[7:0];
        endcase
    end
end
```

Listing 4　I/O 出力デコーダ

タイマの CTRL/LIMIT レジスタへの書き込みは、タイマモジュール自体の we_ctrl、we_limit 入力で直接処理される。

4　7 セグメント表示の改善

4.1　8 桁対応

基本設計では 4 桁（16 ビット）の HEX 表示のみであった。OUT 命令のポート 0（下位 16 ビット）とポート 1（上位 16 ビット）を組み合わせることで、8 桁（32 ビット）のフル HEX 表示を実現する。

4.1.1　桁選択の拡張

桁選択信号を 2 ビットから 3 ビットに拡張し、8 桁を巡回させる。

```
reg [2:0] dig_sel;  // 0~7 の8桁
always @(posedge clk or negedge rst_n) begin
    if (~rst_n)       dig_sel <= 3'd0;
    else if (disp_en) dig_sel <= dig_sel + 3'd1;
end
```

Listing 5　8 桁ダイナミック点灯

分周比は 4kHz (DISP_DIV=5000) を維持し、8 桁巡回のリフレッシュレートは 4000/8 = 500Hz となる。これは人間の目にちらつきとして認識されない十分な周波数である。

4.2 表示モード切替

ロータリースイッチの下位 2 ビットで 4 つの表示モードを切り替える。

表 6 7 セグ表示モード

モード	表示内容
0	OUT レジスタ値 (上位 16bit + 下位 16bit)
1	PC 値 (上位 4 桁=0000、下位 4 桁=PC)
2	レジスタ r0~r3 の下位 8 ビット（各 2 桁 × 4 本 = 8 桁）
3	タイマ COUNT 値

```
wire [1:0] disp_mode = rotsw[1:0];
reg [31:0] disp_data;
always @(*) begin
    case (disp_mode)
        2'd0: disp_data = {seg_reg_hi, seg_reg_lo};
        2'd1: disp_data = {16'h0000, debug_pc};
        2'd2: disp_data = {debug_r3[7:0], debug_r2[7:0],
                           debug_r1[7:0], debug_r0[7:0]};
        2'd3: disp_data = {16'h0000, timer_count};
    endcase
end
```

Listing 6 表示データ選択

4.3 デバッグポート

表示モード 1・2 で使用する CPU 内部信号（PC、レジスタ値）を外部から観測するため、simple_cpu.v にデバッグポートを追加した。

```
output [15:0] debug_pc,
output [15:0] debug_r0, debug_r1, debug_r2, debug_r3,

assign debug_pc = pc;
assign debug_r0 = rf.regs[0];  // 階層参照
assign debug_r1 = rf.regs[1];
assign debug_r2 = rf.regs[2];
assign debug_r3 = rf.regs[3];
```

Listing 7 デバッグポートの実装

5 応用プログラム: バブルソート

5.1 概要

担当 A が追加した ISA 拡張命令（ADDI, BAL/BR）と基本命令を活用し、メモリ上のデータ配列をバブルソートするプログラムを作成した。ソート完了後、結果を昇順に OUT 命令で出力する。

5.2 アルゴリズム

1. メモリアドレス 0x80〜0x8F に 16 個のテストデータを格納: {15, 3, 8, 1, 12, 5, 10, 2, 14, 7, 11, 4, 13, 6, 9, 0}
2. 外側ループ: `i = 15` から `1` まで
3. 内側ループ: `j = 0` から `i-1` まで
4. `data[j] > data[j+1]` なら交換
5. ソート完了後、`data[0]`〜`data[15]` を順に OUT

5.3 レジスタ割り当て

表 7 バブルソートのレジスタ割り当て

レジスタ	用途
r0	ベースアドレス (0x80 固定)
r1	アドレス計算用一時レジスタ
r2	内側ループカウンタ j
r3	外側ループ上限 i
r4	data[j] (比較・交換用)
r5	data[j+1] (比較・交換用)
r6	表示カウンタ / 定数
r7	(予備)

5.4 プログラムリスティング

```
; ==== バブルソート ====
        LI   r3, 15          ; i = 15
outer:  LI   r2, 0           ; j = 0
inner:  MOV  r1, r0          ; r1 = ベースアドレス
        ADD  r1, r2          ; r1 = base + j
        LD   r4, 0(r1)       ; r4 = data[j]
        LD   r5, 1(r1)       ; r5 = data[j+1]
        CMP  r4, r5          ; data[j] - data[j+1]
        BLE  skip             ; data[j] <= data[j+1] ならスワップ不要
```

```
        ST   r5, 0(r1)      ; swap: data[j] = data[j+1]
        ST   r4, 1(r1)      ;       data[j+1] = data[j]
skip:   ADDI r2, 1          ; j++
        CMP  r2, r3         ; j < i ?
        BLT  inner           ; yes -> 内側ループ継続
        ADDI r3, -1         ; i-- (ADDIはフラグ更新)
        BNE  outer           ; i != 0 -> 外側ループ継続
```

Listing 8　バブルソートプログラム (sort.asm) 抜粋 — ソート本体

5.4.1　実装のポイント

- **ADDI 活用**: ループカウンタのインクリメント・デクリメントに ADDI 命令を使用し、従来の LI + ADD の 2 命令を 1 命令に削減。
- **ADDI のフラグ更新**: 外側ループの終了判定で、`ADDI r3, -1` が設定する Z フラグを直接 `BNE` で参照。i=1 のとき ADDI で r3=0 となり Z=1 となるため、BNE が不成立になりループ脱出する。別途 CMP 命令が不要。
- **メモリアクセス**: LD/ST 命令のベースレジスタ + 変位機能を活用。`LD r4, 0(r1)` と `LD r5, 1(r1)` で隣接要素を効率的に読み出す。

5.5　プログラムサイズと実行サイクル数

表 8　バブルソートプログラムの統計

項目	値
プログラムサイズ	58 語 (116 バイト)
データ初期化部	33 語 (LI+ST × 16 + LI r0)
ソート本体	15 語
結果表示部	10 語

5.5.1　実行サイクル数の見積もり

内側ループ 1 回あたり: 11 命令 × 4 サイクル = 44 サイクル。バブルソートの比較回数: $\sum_{i=1}^{15} i = 120$ 回。ソート部の総サイクル数: 約 $120 \times 44 = 5{,}280$ サイクル。20MHz 動作で約 264 μs。

初期化部 (33 命令) + 表示部 (最大 $16 \times 7 = 112$ 命令) を含む全体: 約 $5{,}280 + 132 + 448 = 5{,}860$ サイクル $\approx 293\,\mu$s。

5.6　シミュレーション結果

Icarus Verilog によるシミュレーション結果:

```
=== Bubble Sort Test ===
OUT[0] = 0000  OK
OUT[1] = 0001  OK
```

```
OUT[2] = 0002  OK
OUT[3] = 0003  OK
OUT[4] = 0004  OK
OUT[5] = 0005  OK
OUT[6] = 0006  OK
OUT[7] = 0007  OK
OUT[8] = 0008  OK
OUT[9] = 0009  OK
OUT[10] = 000a  OK
OUT[11] = 000b  OK
OUT[12] = 000c  OK
OUT[13] = 000d  OK
OUT[14] = 000e  OK
OUT[15] = 000f  OK
=== CPU Halted (PC=003a, 16 outputs) ===
=== SORT TEST PASSED ===
```

Listing 9　バブルソートのシミュレーション出力

16 個の要素が 0x0000〜0x000F の昇順に正しくソートされており、プログラムが正常に動作することが確認された。

6　アセンブラ

6.1　概要

SIMPLE プロセッサの全命令（拡張命令 BAL/BR/ADDI 含む）に対応した Python アセンブラ（simple_asm.py）を作成した。ラベル、コメント、PC 相対分岐のアドレス自動計算をサポートする。

6.2　対応命令

表 9　アセンブラ対応命令一覧

分類	ニーモニック	アセンブラ記法	エンコーディング
演算	ADD, SUB, AND, OR	ADD Rd, Rs	形式 (a)
	XOR, CMP, MOV	CMP Rd, Rs	
シフト	SLL, SLR, SRL, SRA	SLL Rd, d	形式 (a)
入出力	IN, OUT	IN Rd, port / OUT Rs, port	形式 (a)
停止	HLT	HLT	形式 (a)
メモリ	LD, ST	LD Ra, d(Rb)	形式 (b)
即値	LI	LI Rb, imm	形式 (c)
分岐	B	B label	形式 (c)
	BE, BLT, BLE, BNE	BLT label	形式 (d)
拡張	BAL	BAL Rb, label	形式 (c), op2=001
	BR	BR Rb	形式 (c), op2=010
	ADDI	ADDI Rb, imm	形式 (c), op2=011

6.3 アセンブラの機能

- **2 パスアセンブル**: パス 1 でラベルアドレスを収集し、パス 2 で機械語を生成
- **ラベル**: 任意の識別子にコロンを付けてラベル定義（例: `loop:`）
- **コメント**: セミコロン以降を無視（例: `; コメント`）
- **PC 相対自動計算**: 分岐命令（B, Bcc, BAL）ではラベルから PC 相対変位を自動計算
- **I/O ポート**: IN/OUT 命令で第 2 オペランドにポート番号を指定可能（省略時はポート 0）
- **リスティング出力**: `-l` オプションでアドレス・機械語・ソースの対応表を表示
- **数値リテラル**: 10 進数、16 進数（0x 接頭辞）、2 進数（0b 接頭辞）に対応

6.4 使用方法

```
$ python3 simple_asm.py sort.asm -o sort.hex -l
0000: 8080  LI   r0, 0x80       ; base address
0001: 810F  LI   r1, 15
0002: 4800  ST   r1, 0(r0)
...
0039: C0F0  HLT
assemble ok: 58 words -> sort.hex
```

Listing 10　アセンブラの使用例

7 動作検証

7.1 テスト環境

表 10　テスト環境

項目	内容
シミュレータ	Icarus Verilog 11.0
テストベンチ	`simple_tb.v` (`TEST_MODE` パラメータで切替)

7.2 ISA 全命令テスト

担当 A のテストプログラム（`test_all.hex`）により、CPU 変更（I/O ポート出力追加）後も全命令が正常に動作することを確認した。

```
=== 4-Cycle CPU + ISA Extension Test ===
OUT[0] = 0008  OK
OUT[1] = 0003  OK
OUT[2] = 000a  OK
OUT[3] = 0014  OK
```

```
OUT[4] = ffff  OK
=== CPU Halted (PC=0012, 5 outputs) ===
=== ALL TESTS PASSED ===
```

Listing 11　ISA テスト結果

7.3　バブルソートテスト

sort.hex によるバブルソートテストでは、16 個の無作為な初期データが昇順に正しくソートされることを確認した（第 5 節のシミュレーション結果を参照）。

7.4　テストベンチの設計

テストベンチはプリプロセッサマクロ（TEST_MODE, INIT_FILE）でテスト対象を切り替える設計とした。

```
# ISA全命令テスト
iverilog -o sim.out simple_tb.v simple_cpu.v alu.v \
    shifter.v regfile.v ram_sim.v && vvp sim.out

# バブルソートテスト
iverilog -DTEST_MODE=1 -DINIT_FILE=\"sort.hex\" -o sim.out \
    simple_tb.v simple_cpu.v alu.v shifter.v regfile.v \
    ram_sim.v && vvp sim.out
```

Listing 12　テストの実行コマンド

8　まとめ

担当 B として、以下の設計・実装を完了した。

1. **タイマモジュール**: プリスケーラ付き 16 ビットハードウェアタイマを設計。20MHz クロックを最大 256 分周し、カウント到達による overflow フラグ機能を実装。
2. **I/O アドレス空間拡張**: IN/OUT 命令の d[3:0] フィールドを I/O ポート番号として利用し、6 ポートのデバイスマッピングを実現。CPU への変更は出力信号 3 本の追加のみで、後方互換性を維持。
3. **7 セグメント表示改善**: 4 桁から 8 桁への拡張と、ロータリースイッチによる 4 モード表示切替（OUT レジスタ/PC/汎用レジスタ/タイマ）を実装。
4. **応用プログラム**: ADDI 命令と LD/ST 命令を活用したバブルソートプログラムを作成。16 要素のソートが 58 語のプログラムで実現でき、シミュレーションで正常動作を確認。
5. **アセンブラ**: 全命令対応の Python アセンブラを作成。ラベル・コメント・PC 相対分岐の自動計算をサポートし、アセンブリプログラムの開発効率を大幅に向上。

担当 A の 4 サイクル CPU・ISA 拡張と統合することで、タイマによる正確な時間計測、複数 I/O デバイスの接続、デバッグ支援機能を備えた実用的なプロセッサシステムが完成した。